Personal Learning Journey

IT3091 Machine Learning - Group 2026-AI-05

DataCo Smart Supply Chain: late-delivery risk

Name: Kodagoda D.S. (Sawanya)

IT number: IT24101613

Area of responsibility: Exploratory data analysis and baseline model

My role

I contributed to the exploratory data analysis and then owned the baseline model. My job was to establish what a simple, honest model achieves, so that everything the rest of the team built afterwards could be measured against something fair.

What I did

On the EDA side I worked on the breakdowns of delay rate by shipping mode, market, region and customer segment, and on checking the data for missing values, duplicates and outliers.

For the modelling I set up three reference points rather than one. A majority-class classifier that always predicts late, a rule that uses only the four Shipping Mode columns, and then Logistic Regression as the real baseline. I also built the evaluation function that every later model reuses, so all seven models are scored the same way on six metrics.

What I learned

The clearest lesson was that accuracy can lie. 57% of our shipped orders are late, so a model that always says late is 57% accurate while knowing nothing at all. Its ROC-AUC is exactly 0.50. Seeing those two numbers next to each other made it obvious why we had to report precision, recall, F1 and ROC-AUC instead.

I also learned what a baseline is actually for. I had thought of it as something you beat, but it is really a measuring stick. Logistic Regression reached 0.7471 and the best model we tried reached 0.7704, so the whole of the rest of the project bought about 0.023. Knowing that early changed how we talked about our results.

Adding the Shipping Mode rule was the part I am most pleased with. It reaches 0.7387 using four columns, which a manager could run in a spreadsheet. It forces the honest question of whether the machine learning is earning its place.

What I found difficult

Understanding why the preprocessing pipeline has to sit inside the cross-validation pipeline took me a while. My first instinct was to transform the data once and then cross-validate, which looks fine and runs faster. It took working through it to see that it fits the scaler and encoder on the validation folds too, so the scores come out slightly too good for a reason that is invisible in the output.

I also found it difficult to resist tuning the baseline. It felt like I was giving it a handicap by leaving it on defaults, but a baseline that has been optimised is no longer a baseline.

A mistake I caught

Early on our expectation was that the tree models would barely improve on Logistic Regression. The first Decision Tree I ran came out at 0.7673, which is about two points better, so that expectation was wrong and we corrected it before writing it into the report.

What I would do differently

I would write the evaluation function before training anything. I wrote the first two models with ad-hoc metric calls and then had to go back and standardise them, which cost time and risked inconsistency between models.

On using AI

I used Claude to help write the modelling code and to explain concepts like why ROC-AUC is threshold-free. I made a point of running everything myself and checking the numbers against the saved result files rather than trusting what appeared on screen.